# DevOps Case Study & Ansible Automation

FoodApp: automated deployment of an online food-ordering web server (Nginx) with Ansible

| Field | Details |
|---|---|
| Student name | Atharva Suryawanshi |
| PRN / ID | 202301040283 |
| Course | **Cloud Native Devops (MDM)** |
| Instructor | Dr. Navnath Kale |
| Assignment | DevOps Case Study & Ansible Automation Assignment |
| Demo video | Click for Link |
| Submission date | 14 October 2026 |

## 1. Introduction and Scenario

This case study uses **FoodApp**, an online food-ordering application. Customers open a web front end, browse the menu and place orders; a backend and database store the orders. The application is served to users by an Nginx web server running on Linux.

Food ordering is a good DevOps example. Demand peaks at lunch and dinner, so servers must be added quickly and must be configured identically. New features and menu changes are released often, and a failed deployment during a rush means lost orders. Setting up a web server by hand for every environment (development, test, production) is slow and error-prone. This project analyses the DevOps lifecycle for FoodApp and implements the **Deploy** and **Operate** stages with Ansible, so that a server can be installed, configured and verified automatically, repeatably and safely.

### 1.1 Objectives

- Analyse the DevOps lifecycle, workflow and architecture of the application.
- Identify the practices and automation requirements at each stage.
- Prepare an Ansible inventory with host groups and variables.
- Develop a reusable playbook (variables, templates, handlers, tags) that installs and configures Nginx and deploys the application page.
- Demonstrate successful, idempotent execution with screenshots and explain the concepts used.

## 2. DevOps Lifecycle for FoodApp

DevOps joins development and operations in a continuous loop in which monitoring results flow back into planning. Figure 1 shows the eight stages selected for FoodApp, and Table 1 lists the activity, tools and status of each stage in this project.

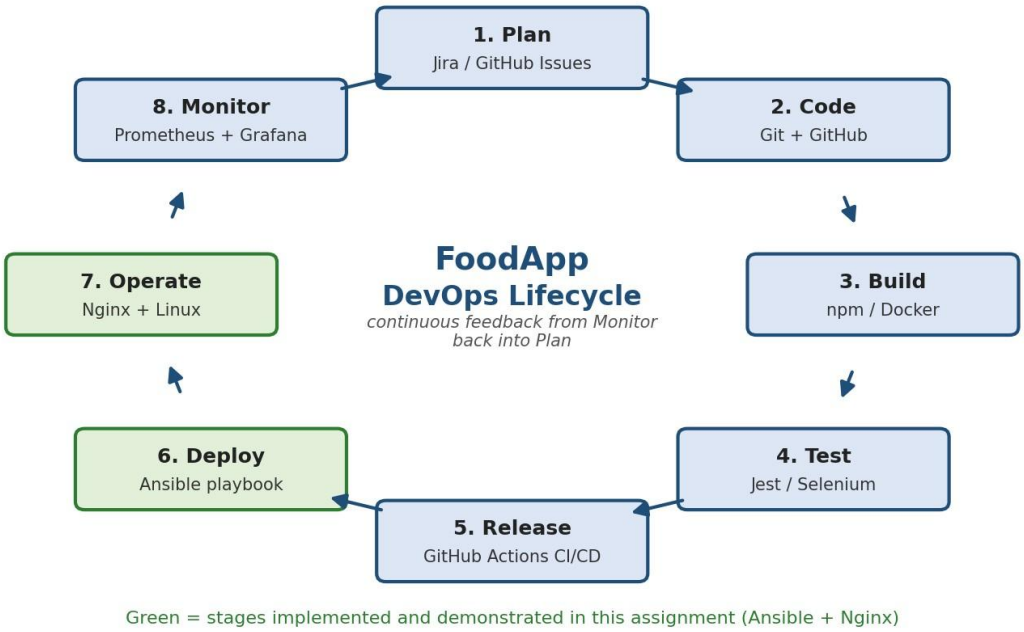



*Figure 1: DevOps lifecycle for FoodApp (Deploy and Operate implemented with Ansible and Nginx)*

| Stage | Activity in FoodApp | Tools | Status in this project |
|---|---|---|---|
| 1. Plan | Backlog of features (menu, cart, order tracking) and bug tickets; sprint planning | Jira / GitHub Issues | Designed |
| 2. Code | Application code and Ansible files version-controlled; branches and pull requests | Git, GitHub | Designed |
| 3. Build | Package the application and its dependencies reproducibly | npm, Docker | Designed |
| 4. Test | Automated unit and integration tests; syntax check of playbooks | Jest, Selenium, ansible-playbook --syntax-check | Designed; syntax check executed |
| 5. Release | CI pipeline tests every push, then triggers the deployment job | Jenkins / GitHub Actions | Designed |
| 6. Deploy | Configure the web server identically on every host | **Ansible** | Implemented and demonstrated |
| 7. Operate | Run the web server as a managed service that starts at boot | Nginx, Linux (systemd) | Implemented and demonstrated |
| 8. Monitor | Metrics, logs and alerts; feedback into Plan | Prometheus, Grafana | Designed |

*Table 1: Lifecycle stages, tools and project status.*

# 3. Workflow and Architecture

## 3.1 Workflow

- **Commit:** a developer pushes application or playbook changes to the GitHub repository.
- **Integrate:** a CI pipeline builds and tests the code automatically and fails fast if anything is broken.
- **Deploy:** if the tests pass, the Ansible control node runs `site.yml` against the inventory, installing and configuring Nginx and deploying the application page.
- **Verify:** the playbook ends by requesting the site on port 8080, so a broken deployment is detected immediately.
- **Operate and monitor:** systemd keeps Nginx running; monitoring tools watch performance and errors, and issues feed back into planning.

## 3.2 Architecture

Figure 2 shows the architecture. The Ansible control node reads `inventory.ini`, which defines the group *foodapp* containing the child group *webservers* with the host *web1*. On the managed node, Nginx listens on port 8080 and serves the application page from `/var/www/foodapp`. In the demonstration the control node and the managed node are the same Ubuntu (WSL2) machine, reached with `ansible_connection=local`; the same playbook works unchanged over SSH for remote servers.

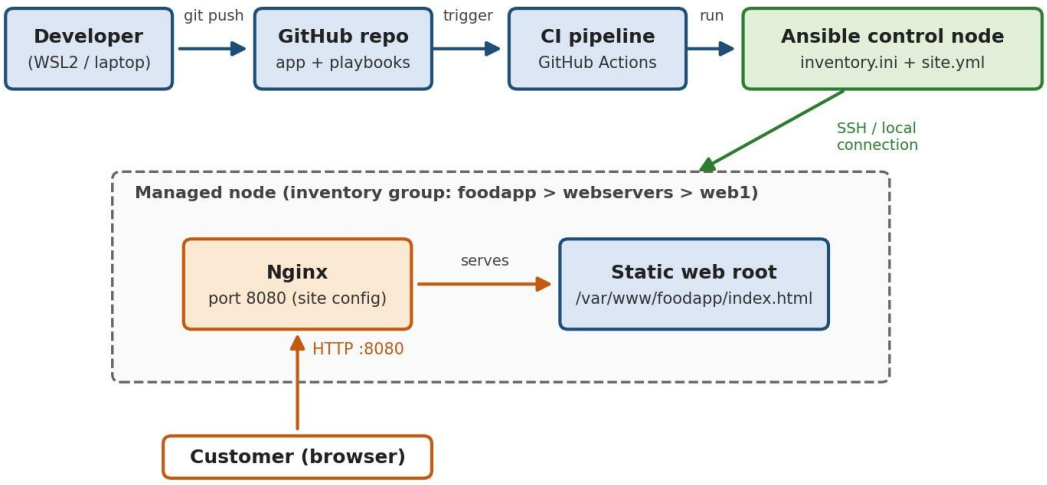



*Figure 2: Deployment architecture and workflow*

## 3.3 DevOps practices applied

| Practice | How it is applied in FoodApp |
|---|---|
| Infrastructure as Code | Server setup lives in YAML and template files that can be reviewed, versioned and re-run instead of manual commands. |
| Configuration management | Ansible enforces the desired state: package installed, directories, site configuration, enabled site, running service. |

| | |
|---|---|
| Idempotency | Running the playbook again changes nothing when the server is already correct (Section 6). |

| Practice | How it is applied in FoodApp |
|---|---|
| Continuous integration / delivery | Pipeline designed to build, test and then trigger the Ansible deployment on every push. |
| Verification | A final task checks that the site returns HTTP 200 after deployment. |
| Monitoring and feedback | Metrics and logs are collected and fed back into planning (designed). |

# 4. Automation Requirements

From the lifecycle analysis, the deployment stage must automate the following, because each item is repetitive and a manual mistake would cause downtime:

- Install the required operating-system packages (Nginx) on every server.
- Create the web root directory with the correct permissions.
- Deploy the application page and the web-server configuration from templates, filled with variables.
- Enable the application site and disable the default Nginx site.
- Start the service now and enable it at boot; reload it only when its configuration changes.
- Verify automatically that the application answers on its port.
- Keep many servers identical by running the same playbook against all of them.

The benefits are that deployment becomes **repeatable, consistent, fast and idempotent**, and human error is greatly reduced. Ansible was chosen because it is agentless (only SSH and Python are needed on the targets), uses readable YAML, is idempotent by design and supports templates and handlers for reuse. Puppet and Chef require an agent on every node.

# 5. Ansible Implementation

## 5.1 Project structure

**devops-assignment/**

```
devops-assignment/ |-- inventory.ini
# hosts and groups
|-- site.yml                # the playbook
`-- templates/
    |-- nginx.conf.j2       # Nginx site configuration template
    `-- index.html.j2       # application page template
```

## 5.2 Inventory

The inventory defines the group *webservers* with one host, *web1*, and a parent group *foodapp* that contains it, so shared settings are written once under `[foodapp:vars]`. The host uses a local connection, so no SSH is needed for the demonstration.

**inventory.ini**

```
[webservers] web1 ansible_host=127.0.0.1
ansible_connection=local

[foodapp:children]
webservers

[foodapp:vars]
ansible_python_interpreter=/usr/bin/python3
```

## 5.3 Playbook

`site.yml` contains one play for the *webservers* group with privilege escalation (`become: true`). Variables (`app_name`, `app_port`, `web_root`) keep values out of the tasks. Each task carries a tag, so parts of the play can be run separately (for example `--tags config`). Three handler notifications trigger a single reload of Nginx, and the final task verifies the result.

**site.yml (part 1 of 2)**

```
--- name: Configure FoodApp web
server   hosts: webservers   become:
true   vars:
    app_name: foodapp
app_port: 8080     web_root:
/var/www/foodapp

  tasks:     - name:
Install Nginx       apt:
        name: nginx
state: present
update_cache: yes
cache_valid_time: 3600
tags: packages

-       name: Create web root directory      file:
        path: "{{ web_root }}"
state: directory         mode:
"0755"       tags: app

-       name: Deploy application page from
template      template:
        src: index.html.j2         dest:
"{{ web_root }}/index.html"         mode:
"0644"       tags: app
```

**site.yml (part 2 of 2)**

```
-       name: Deploy Nginx site configuration
template:
        src: nginx.conf.j2          dest:
"/etc/nginx/sites-available/{{ app_name }}"          mode:
"0644"        notify: Reload nginx        tags: config

-       name: Enable FoodApp site       file:
        src: "/etc/nginx/sites-available/{{ app_name }}"
dest: "/etc/nginx/sites-enabled/{{ app_name }}"
state: link        notify: Reload nginx        tags:
config

-       name: Disable default Nginx site
file:
        path: /etc/nginx/sites-enabled/default
state: absent        notify: Reload nginx
tags: config

-       name: Ensure Nginx is running and
enabled at boot       service:
        name: nginx
state: started
enabled: yes        tags:
service

-       name: Apply pending handlers now
meta: flush_handlers

-       name: Verify the application responds
uri:
        url: "http://localhost:{{ app_port }}"
status_code: 200        tags: verify

  handlers:     - name:
Reload nginx
service:
        name: nginx
state: reloaded
```

## 5.4 Templates

The two Jinja2 templates are filled with the playbook variables and Ansible facts when they are deployed. **templates/nginx.conf.j2**

```
server {
    listen {{ app_port }};
server_name _;      root {{
web_root }};      index
index.html;

    location / {          try_files
$uri $uri/ =404;      }
}
```

**templates/index.html.j2**

```
<!DOCTYPE html>
<html>
<head><title>{{ app_name }}</title></head>
<body style="font-family: Arial; text-align: center; margin-top: 80px;">
  <h1>Welcome to {{ app_name }} - Online Food Ordering</h1>
  <p>Deployed automatically using Ansible</p>
  <p>Server: {{ ansible_hostname }}</p>
```

```
  <p>Student: Atharva Suryawanshi - 202301040283</p>
</body>
</html>
```

## 5.5 Task summary

| Task | Module | Tag | Purpose |
|---|---|---|---|
| Install Nginx | apt | packages | Installs the web server (cache refreshed at most hourly) |
| Create web root directory | file | app | Creates /var/www/foodapp with mode 0755 |
| Deploy application page | template | app | Renders index.html from index.html.j2 |
| Deploy Nginx site configuration | template | config | Renders the site file; notifies the handler |
| Enable FoodApp site | file (link) | config | Symlinks the site into sites-enabled |
| Disable default Nginx site | file (absent) | config | Removes the default site so port 8080 serves FoodApp |
| Ensure Nginx is running and enabled | service | service | Starts Nginx now and at boot |
| Apply pending handlers now | meta: flush_handlers | - | Reloads Nginx before the verification task |
| Verify the application responds | uri | verify | Requests http://localhost:8080 and expects HTTP 200 |

*Table 2: Tasks in site.yml.*

## 5.6 Reusability and good practice

- **Variables** keep names, ports and paths out of the tasks, so the same playbook works for another application or port.
- **Templates** generate the page and the Nginx site from variables and facts such as the host name.
- **Handlers** reload Nginx only when a configuration task actually changed something.
- **flush_handlers** applies the pending reload before the verification task. Without it, the final check could run before Nginx had reloaded and fail.
- **Tags** allow partial runs (packages, app, config, service, verify).
- **Built-in modules** (apt, file, template, service, uri) are idempotent, so repeated runs are safe.

# 6. Execution and Results

The playbook was run on Ubuntu (WSL2 with systemd) using the local inventory, from the project folder in the Linux home directory. Every screenshot shows the student name and PRN and the date, printed with `echo` and `date` at the top of the terminal. Commands used:

**Commands**

```
ansible --version ls -R cat inventory.ini ansible-
inventory -i inventory.ini --graph ansible all -i
inventory.ini -m ping ansible-playbook -i inventory.ini
site.yml --syntax-check ansible-playbook -i inventory.ini
site.yml -K curl http://localhost:8080 systemctl status
nginx
```

## 6.1 Environment, project files and inventory

Ansible core 2.16.3 is installed with Python 3.12.3. The project contains the inventory, the playbook and the two templates, and the inventory file holds the groups and variables described in Section 5.

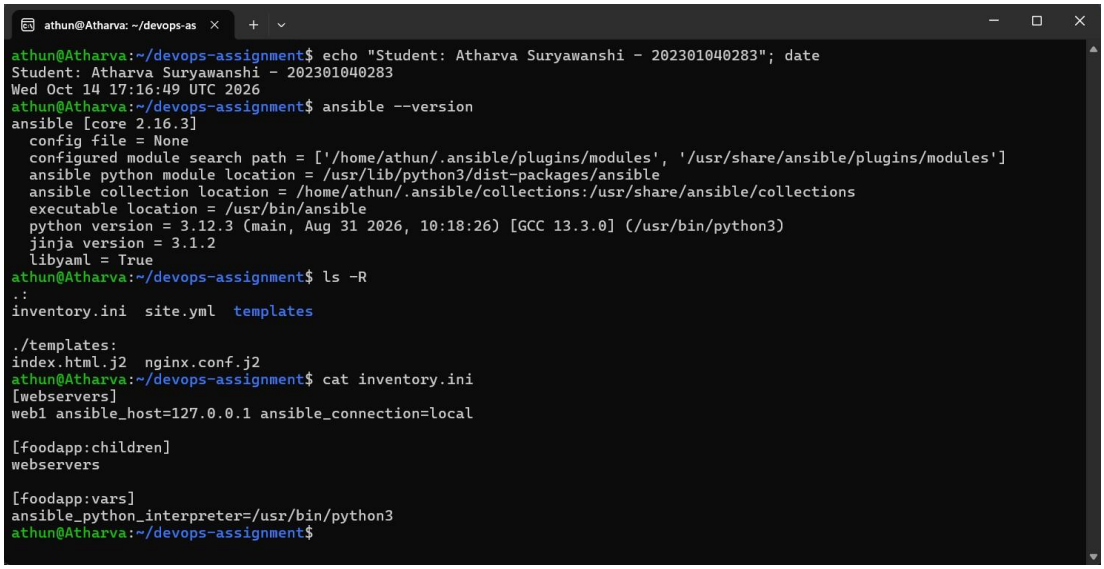



*Figure 3: ansible --version, ls -R and cat inventory.ini (Ansible core 2.16.3; project files present)*

## 6.2 Inventory graph, connectivity and syntax check

The inventory graph shows the host *web1* under *webservers*, which is under *foodapp*. The ping module returns `pong`, proving Ansible can manage the host, and the syntax check passes before any change is made.

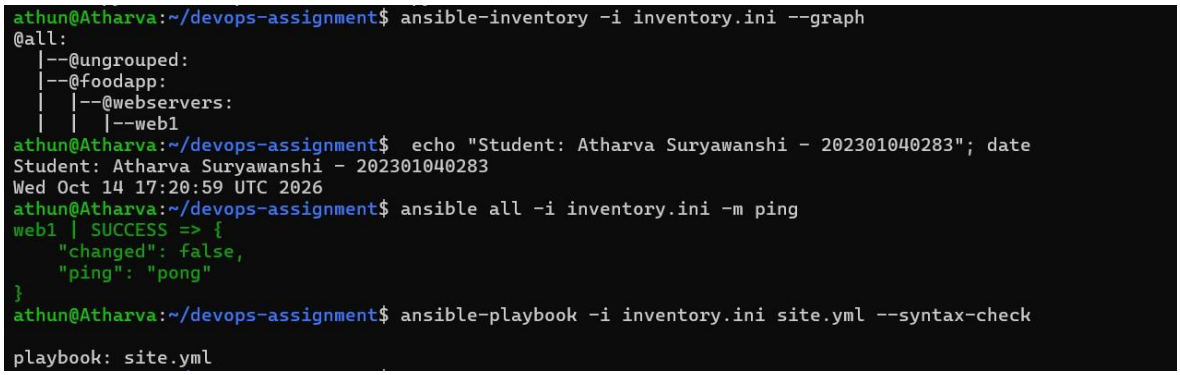


*Figure 4: Inventory graph, ansible ping (SUCCESS, pong) and --syntax-check (playbook: site.yml)*

## 6.3 First run: configuration applied

The first run changed seven items: Nginx installed, web root created, page deployed, site configuration deployed, site enabled, default site disabled, and the Nginx reload handler executed. The service task was already *ok* because the Nginx package starts the service on installation, and the verification task returned HTTP 200. The recap was **ok=10 changed=7 unreachable=0 failed=0**.

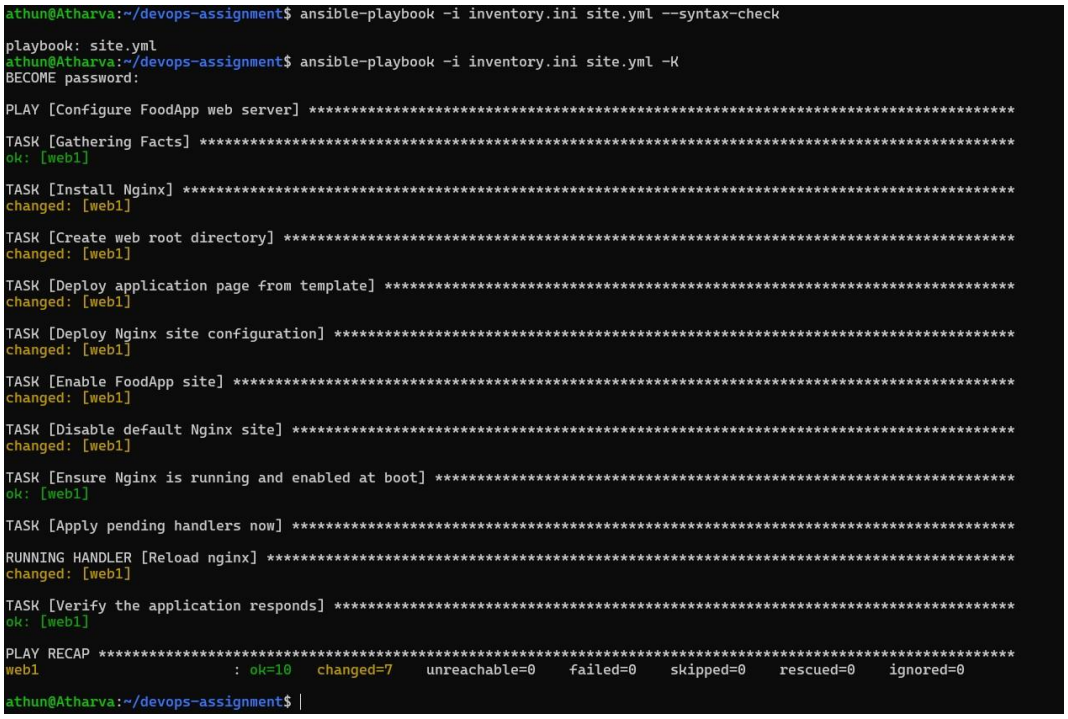



*Figure 5: First run (-K prompts for the sudo password): PLAY RECAP ok=10 changed=7 failed=0*

## 6.4 Second run: idempotency

Running the same playbook again reported **changed=0** and no failures. Every task was already in the desired state and the handler did not run, which proves the playbook is idempotent.

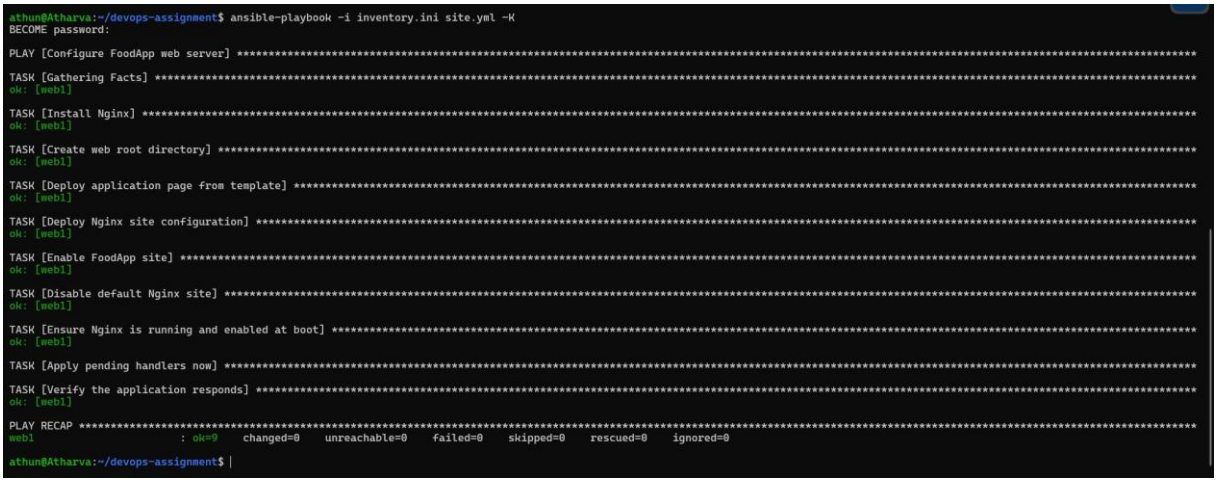



*Figure 6: Second run: PLAY RECAP ok=9 changed=0 failed=0 (idempotent)*

| Run | ok | changed | failed | Meaning |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| 1 (first run) | 10 | 7 | 0 | Nginx installed, configured and the site deployed; handler reloaded Nginx. |
| 2 (repeat) | 9 | 0 | 0 | Desired state already met; nothing changed (idempotent). |

*Table 3: Summary of the two runs.*

## 6.5 Verification

The page was requested with `curl` on port 8080 and returns the HTML rendered from the template, including the host name and student details. systemd shows Nginx as *enabled* and *active (running)*, with the reload recorded in the journal. Finally, the page opens in the Windows browser.

```
athun@Atharva:~/devops-assignment$ curl http://localhost:8080
<!DOCTYPE html>
<html>
<head><title>foodapp</title></head>
<body style="font-family: Arial; text-align: center; margin-top: 80px;">
  <h1>Welcome to foodapp - Online Food Ordering</h1>
  <p>Deployed automatically using Ansible</p>
  <p>Server: Atharva</p>
  <p>Student: Atharva Suryawanshi - 202301040283</p>
</body>
</html>
athun@Atharva:~/devops-assignment$
```

*Figure 7: curl http://localhost:8080 returns the page generated from index.html.j2*

```
athun@Atharva:~/devops-assignment$ systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
     Active: active (running) since Wed 2026-10-14 17:21:51 UTC; 4min 41s ago
       Docs: man:nginx(8)
    Process: 8089 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
    Process: 8091 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
    Process: 8410 ExecReload=/usr/sbin/nginx -g daemon on; master_process on; -s reload (code=exited, status=0/SUCCESS)
   Main PID: 8132 (nginx)
      Tasks: 17 (limit: 9163)
     Memory: 12.0M (peak: 26.8M)
        CPU: 239ms
     CGroup: /system.slice/nginx.service
             ├─8132 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             ├─8411 "nginx: worker process"
             ├─8413 "nginx: worker process"
             ├─8414 "nginx: worker process"
             ├─8415 "nginx: worker process"
             ├─8416 "nginx: worker process"
             ├─8417 "nginx: worker process"
             ├─8418 "nginx: worker process"
             ├─8419 "nginx: worker process"
             ├─8420 "nginx: worker process"
             ├─8421 "nginx: worker process"
             ├─8422 "nginx: worker process"
             ├─8423 "nginx: worker process"
             ├─8424 "nginx: worker process"
             ├─8425 "nginx: worker process"
             ├─8426 "nginx: worker process"
             └─8427 "nginx: worker process"

Oct 14 17:21:51 Atharva systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Oct 14 17:21:51 Atharva systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
Oct 14 17:21:56 Atharva systemd[1]: Reloading nginx.service - A high performance web server and a reverse proxy server...
Oct 14 17:21:56 Atharva nginx[8410]: 2026/10/14 17:21:56 [notice] 8410#8410: signal process started
Oct 14 17:21:56 Atharva systemd[1]: Reloaded nginx.service - A high performance web server and a reverse proxy server.
athun@Atharva:~/devops-assignment$
```

*Figure 8: systemctl status nginx: enabled and active (running); the reload was triggered by the handler*




*Figure 9: The deployed application opened in a browser at localhost:8080*

# 7. Design Decisions and Problems Avoided

| Issue | Cause | Fix |
|---|---|---|
| Final verification could fail right after the config change | The Nginx reload handler runs at the end of the play by default, so the check might run before the new configuration was active. | Added `meta: flush_handlers` before the verification task so the reload happens first. |
| Permission error when running tasks | Package and service tasks need root privileges. | Set `become: true` in the play and run with `-K` to enter the sudo password. |
| Nginx cannot be managed as a service | systemd must be enabled in WSL. | Checked with `systemctl is-system-running` and enabled it in /etc/wsl.conf if needed. |
| Default Nginx site conflicts with the new site | Both would compete for the default configuration. | A task removes the default site from sites-enabled; the FoodApp site is enabled instead. |

*Table 4: Problems considered and how they were resolved.*

# 8. Limitations and Future Work

- Only the Deploy and Operate stages were implemented and executed. Plan, Code, Build, Test, Release and Monitor are described in the design but not built.
- The demonstration used a single machine as both control node and managed node. Adding more hosts to the *webservers* group, with SSH settings, scales the same playbook to many servers.
- The application is a static page. A real FoodApp would also need a backend and a database tier, which could be added as separate roles.
- Future work: a GitHub Actions pipeline that lints and tests the code and then runs the playbook, HTTPS with Let's Encrypt, Ansible roles and Vault for secrets, and Prometheus and Grafana for monitoring.

# 9. Conclusion

This project analysed a realistic application, FoodApp, through the full DevOps lifecycle and automated the deployment of its web tier with Ansible. The inventory, variables, templates, handlers and tags install Nginx, deploy the configuration and the application page, start the service and verify the result. The first run applied seven changes, and the second run reported zero changes, proving idempotency. The assignment showed how configuration management removes manual effort, prevents configuration drift and makes deployments repeatable and reliable.